

GEMMA 4

OLLAMA

LLAMA.CPP

LITERT

UNSLOTH

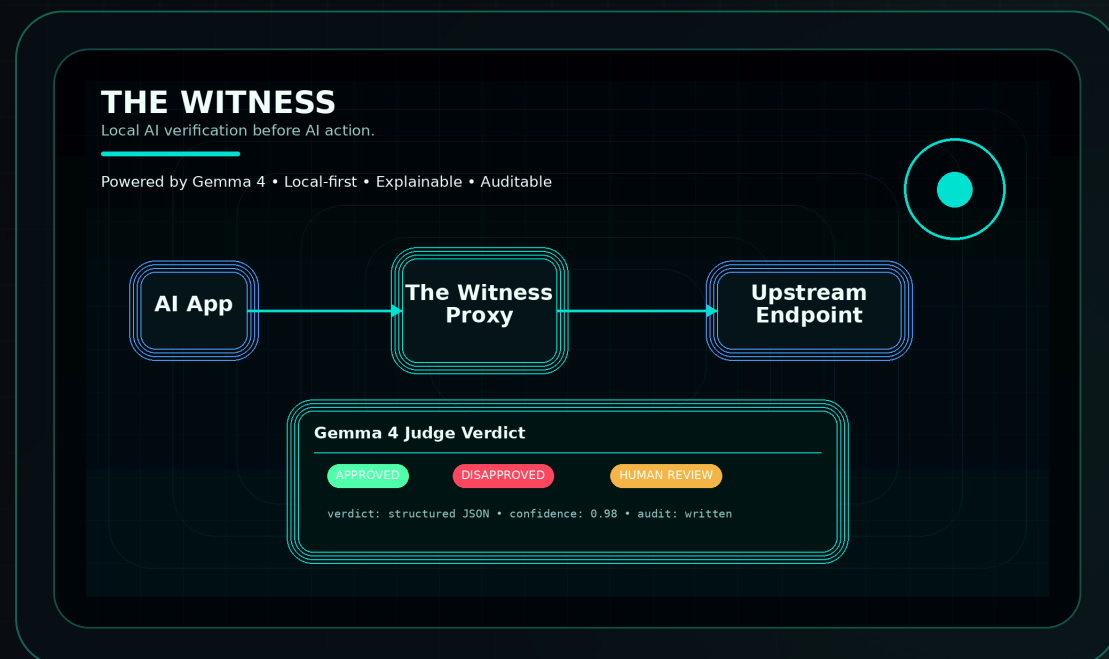
HUGGING FACE

SAFETY & TRUST

The Witness

A Local-First Gemma 4 Reliability Firewall for AI Endpoints

Do not just trust AI. Let The Witness see it first.



GitHub: <https://github.com/PLASMA-FR/the-witness>

Fine-tuned model: <https://huggingface.co/ahmadalfakeh/witness-gemma4-e2b-judge>

Colab notebook: [https://colab.research.google.com/drive/17-](https://colab.research.google.com/drive/17-CgEQLNg8bpnhhWzJwpapRxQyHlqybq?usp=sharing)

[CgEQLNg8bpnhhWzJwpapRxQyHlqybq?usp=sharing](https://colab.research.google.com/drive/17-CgEQLNg8bpnhhWzJwpapRxQyHlqybq?usp=sharing)

NAVIGATION

Table of Contents

[1. Executive Summary](#)[2. The Problem](#)[3. The Solution](#)[4. Core Workflow](#)[5. Features](#)[6. TUI Guide](#)[7. Installation](#)[8. Model Setup](#)[9. Adding an Endpoint](#)[10. Gemma 4 Judge Schema](#)[11. Prompt Repair](#)[12. Technology Tracks](#)[13. Impact Tracks](#)[14. Demo Walkthrough](#)[15. Security and Privacy](#)[16. Limitations](#)[17. Reproducibility](#)[18. Links](#)[19. Final Pitch](#)

Reader promise: this booklet shows the project workflow, the local proxy architecture, the TUI, the Gemma judge schema, setup commands, demo path, tracks, and honest limitations.

OVERVIEW

1. Executive Summary

The Witness is a TUI-based local proxy that lets users add AI endpoints, watch every request and response, verify each response using Gemma 4, block bad answers, repair prompts, retry, and only return approved responses.

“The Witness is not another chatbot.”

It is the verification layer between AI generation and real-world action.

Instead of trusting the first response from an upstream model, The Witness asks a local Gemma 4 judge whether the response is safe, useful, aligned with the prompt, and low-risk enough to pass through.



```
$ the-witness start
```

```
Dashboard → Endpoint Watchlist → Live  
Stream → Gemma Verdict → Prompt Repair  
→ Audit Log
```

WHY IT MATTERS

2. The Problem

AI tools are becoming agents. They write code, tutor students, summarize health information, support finance workflows, and automate work. But many tools trust the first model response immediately.

Incorrect code

A weak answer can create broken scripts, unsafe shell commands, or insecure defaults.

Bad tutoring

Students may see plausible but wrong explanations before anyone reviews them.

Overconfident advice

Health, finance, and legal topics require caution, uncertainty, and human review.

Hallucinated facts

Agents may invent APIs, citations, or operational details.

Untraceable decisions

Without logs, teams cannot reconstruct what was approved, rejected, or retried.

Hidden risk

The failure happens between the model and the user. The Witness puts a gate there.

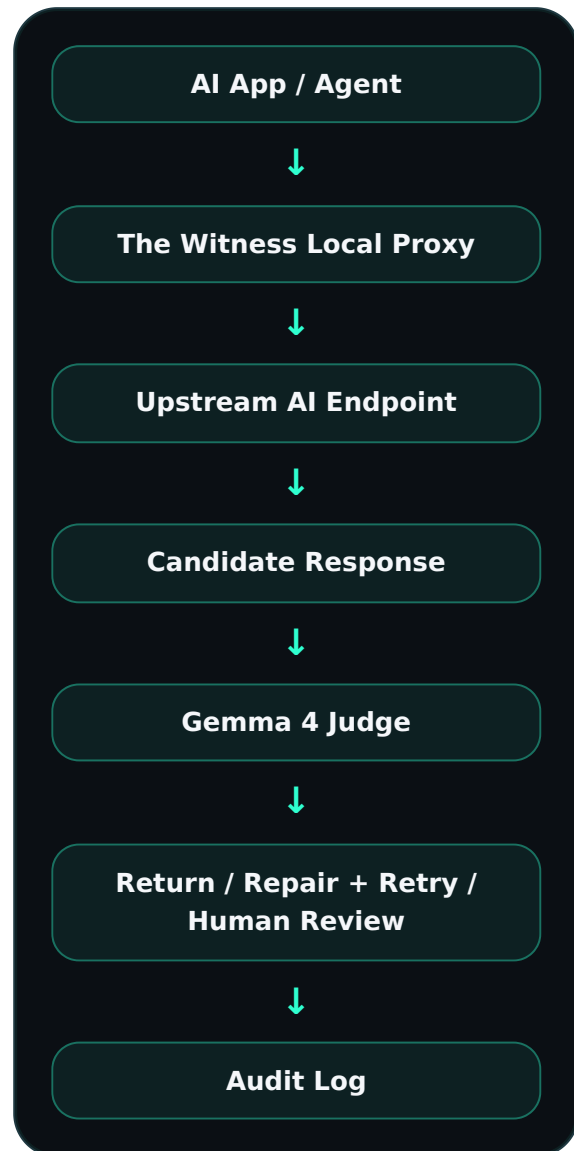
Risk statement: The point is not to claim perfect safety. The point is to add a local, explainable verification gate before model output reaches real users.

ARCHITECTURE

3. The Solution

The Witness sits between the AI app and the upstream model endpoint. The original application calls a localhost OpenAI-compatible proxy. The Witness forwards the request, captures the candidate response, and asks Gemma 4 for a structured verdict before releasing anything.

- watches the endpoint
- captures request/response pairs
- sends candidate responses to Gemma 4
- approves, rejects, repairs, retries, or pauses for human review
- writes an audit trail



USAGE FLOW

4. Core Workflow



1. User adds endpoint.
2. The Witness creates a local proxy URL.
3. AI app sends request to local proxy.
4. The Witness forwards request to upstream endpoint.
5. Upstream returns candidate response.
6. Gemma 4 judges response.
7. If approved, response is returned.
8. If disapproved, response is blocked.
9. Prompt repair is generated.
10. Request is retried.
11. If risky, human review queue is used.
12. Logs are saved.

CAPABILITIES

5. Features

TUI dashboard

Live operational view of endpoint health, request stats, approvals, rejections, retry counts, and current judge configuration.

Why it matters: Judges and users can see the system working without opening a browser.

Endpoint watchlist

Add, edit, enable, disable, test, and duplicate watched endpoint configurations.

Why it matters: Every endpoint can have its own proxy URL, profile, retry limit, strictness, and fallback behavior.

OpenAI-compatible local proxy

Applications point at localhost while The Witness forwards to the real upstream endpoint.

Why it matters: Existing tools can adopt the verification layer with minimal configuration changes.

Request/response monitoring

Captures model name, prompt content, metadata, latency, and candidate response before release.

Why it matters: Turns invisible model traffic into an auditable timeline.

Gemma 4 judge

A local Gemma 4 model reviews candidate responses and returns structured safety and quality verdicts.

Why it matters: Keeps the trust layer local-first and inspectable.

Structured JSON verdicts

The judge must output a strict schema with verdict, confidence, scores, risk, reason, and repair instruction.

Why it matters: Machine-readable decisions make automation and auditing possible.

Approval states

Supports APPROVED, DISAPPROVED, and NEEDS_HUMAN_REVIEW.

Why it matters: Not every response is binary; risky cases can pause for a person.

Prompt repair loop

Rejected answers produce hidden corrective instructions and a retry request.

Why it matters: The app improves outputs instead of simply failing silently.

Automatic retry

Retries until approved or the endpoint retry limit is reached.

Why it matters: Gives upstream models a chance to correct mistakes before users see them.

Human review queue

Risky or uncertain outputs can be approved, rejected, edited, regenerated, exported, or annotated.

Why it matters: Critical workflows get human judgment when needed.

Audit logs

JSONL logs record decisions, retry chains, prompt repairs, and endpoint errors.

Why it matters: Post-incident review and hackathon demos have evidence.

Secret redaction

Authorization headers and API keys are hidden in TUI and logs.

Why it matters: Demos and screenshots remain safe to share.

Privacy mode

Can store metadata only instead of full prompts/responses.

Why it matters: Supports sensitive workflows and low-trust environments.

Per-endpoint profiles

Profiles tune criteria for coding, education, medical, finance, legal, research, disaster response, and multilingual use.

Why it matters: Different domains need different review strictness.

Model manager

Lists Gemma model choices and backends including Ollama, llama.cpp, LiteRT, Unsloth, and manual endpoints.

Why it matters: Users can choose speed, quality, or hardware fit.

Ollama support

Recommended local backend with configurable Gemma model tags such as gemma4:e2b and gemma4:e4b.

Why it matters: The easiest path for local Gemma judging.

llama.cpp support

Connects to local/resource-constrained GGUF inference servers.

Why it matters: Supports CPU/GPU constrained local deployments.

LiteRT edge prefilter

Experimental lightweight edge classification path before full judging.

Why it matters: Enables future low-latency edge safety checks.

Unsloth fine-tuning notebook

Public Colab notebook fine-tunes a Witness Gemma 4 E2B judge with LoRA/QLoRA.

Why it matters: Makes the judge specialization reproducible.

Hugging Face model support

The fine-tuned Witness Gemma 4 E2B judge adapter is published on Hugging Face.

Why it matters: Users can download and inspect the model artifact path.

Blackbox endpoint example

A ready endpoint template uses BLACKBOX_API_KEY through environment variables.

Why it matters: Shows real external endpoint use without committing secrets.

Doctor command

Checks backend, model, schema, proxy, logs, endpoint, and optional integrations.

Why it matters: Pre-demo verification is fast and transparent.

Curl installer

Install path can be scripted while still offering safer download-review-run steps.

Why it matters: Accessible for new users and judges.

Demo mode

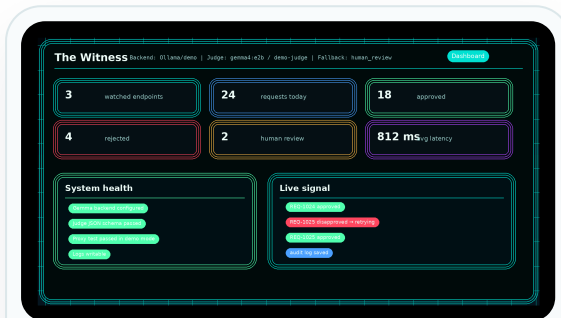
Deterministic local judge and mock/demo endpoint make the workflow visible without external keys.

Why it matters: Hackathon judges can evaluate the idea even without private API credentials.

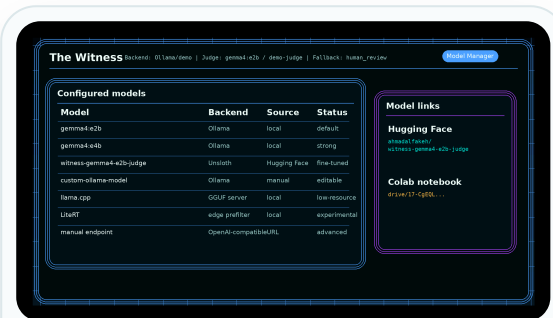
SCREENS

6. TUI Guide

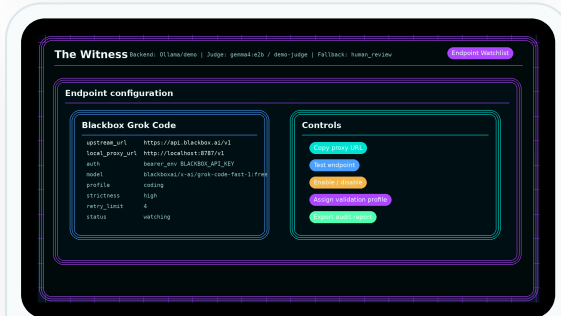
The Witness is intentionally a modern TUI, not a web GUI. It is designed for developers, local-first users, and hackathon judges who want to see actual endpoint traffic and verification decisions.



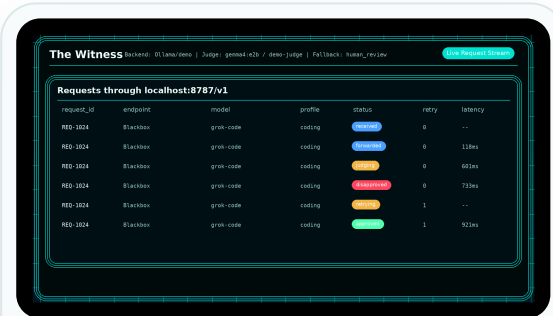
Dashboard: backend/model/fallback status, watched endpoints, request statistics, and system health.



Model Manager: Ollama, Hugging Face fine-tuned model, llama.cpp, LiteRT, and manual endpoints.



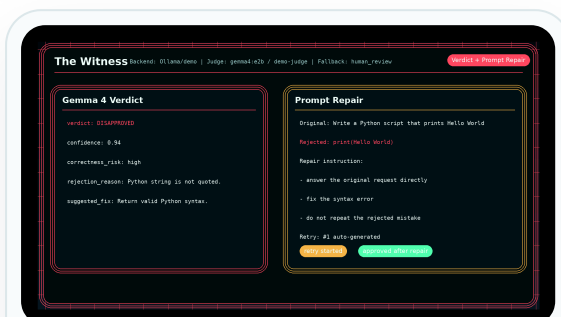
Endpoint Watchlist: local proxy URL, upstream URL, profile, strictness, retry limit, and auth redaction.



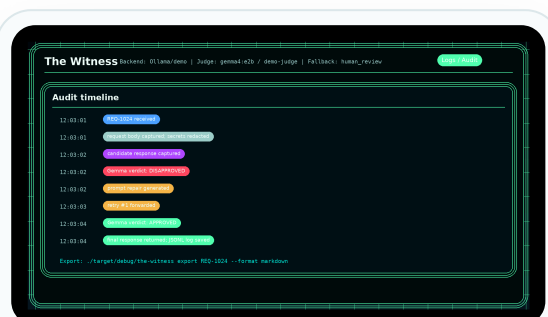
Live Request Stream: received, forwarded, judging, disapproved, retrying, and approved statuses.

SCREENS CONTINUED

7. TUI Deep Dive



Gemma Verdict and Prompt Repair panel: rejection reason, suggested fix, repaired prompt, and retry status.



Logs / Audit screen: full event timeline, rejected/approved events, retry chains, and export path.

Setup Wizard: choose backend, choose model, test judge, test proxy, and pass the readiness checklist before the dashboard opens.

Human Review Queue: approve, reject, edit, regenerate, export a report, mark unsafe, or add a note when the judge is uncertain or the domain is high-risk.

GETTING STARTED

8. Installation

Quick install

```
curl -fsSL https://raw.githubusercontent.com/PLASMA-FR/the-witness/main/scripts/install.sh | bash
```

Safer install

```
curl -fsSL https://raw.githubusercontent.com/PLASMA-FR/the-witness/main/scripts/install.sh -o install.sh  
less install.sh  
bash install.sh
```

Manual install

```
git clone https://github.com/PLASMA-FR/the-witness.git  
cd the-witness  
cargo build --release  
./target/release/the-witness setup  
./target/release/the-witness doctor  
./target/release/the-witness start
```

Requirements: Rust, Cargo, optional Ollama, optional Gemma models, optional Blackbox API key, optional Hugging Face model. Doctor warnings are normal when optional integrations are not configured.

GEMMA AND BACKENDS

9. Model Setup

Default local judge

```
ollama pull gemma4:e2b
```

E2B is the fast/default judge for quick approval classification.

Strong/high-risk judge

```
ollama pull gemma4:e4b
```

E4B is stronger for coding and high-risk profiles when hardware allows.

Fine-tuned Witness judge

<https://huggingface.co/ahmadalfakeh/witness-gemma4-e2b-judge>

```
the-witness model download --source huggingface --model witness-gemma4-e2b-judge
```

Fine-tuning notebook

<https://colab.research.google.com/drive/17-CgEQLNg8bpnhhWzJwpapRxQyHlqybq?usp=sharing>

The notebook fine-tunes a Witness Gemma 4 E2B Judge using Unsloth to output structured JSON verdicts.

Hugging Face model reference

Fine-tuned Witness Gemma 4 E2B judge adapter.

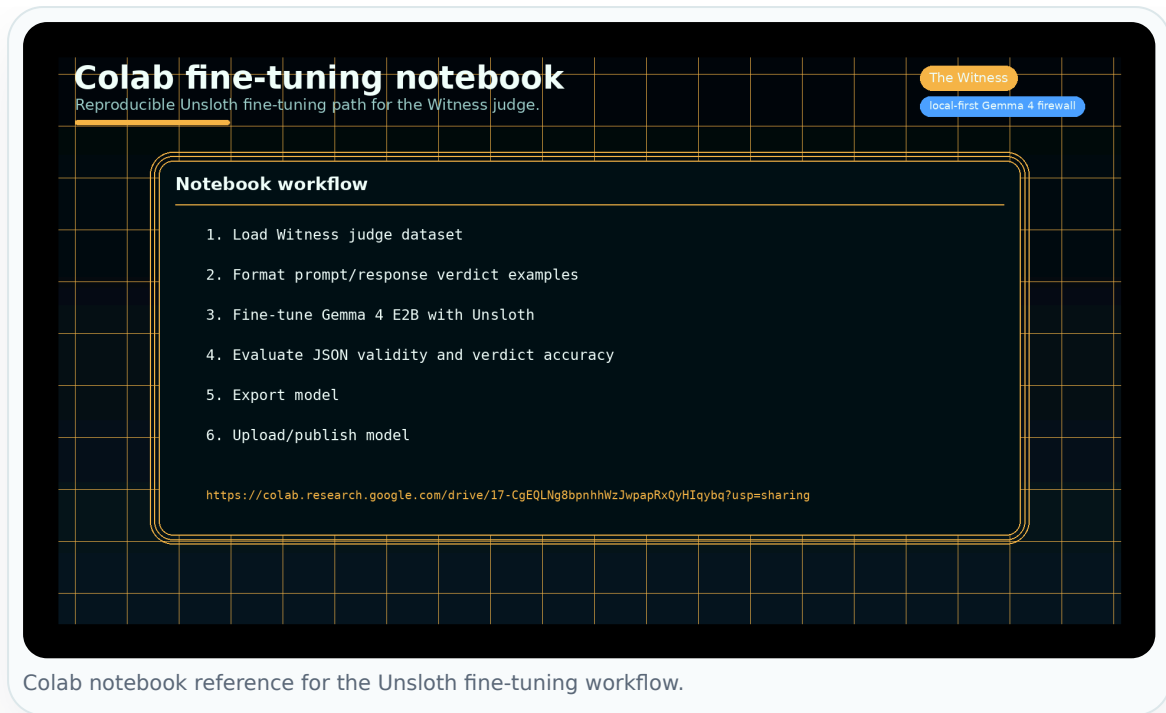
The Witness

local-first Gemma 4 firewall

Fine-tuned model

Model	ahmadalfakeh/witness-gemma4-e2b-judge
Link	https://huggingface.co/ahmadalfakeh/witness-gemma4-e2b-judge
Command	the-witness model download --source huggingface --model witness-gemma4-e2b-judge
Purpose	Structured JSON verdicts for approval, rejection, or human review

Hugging Face reference for the fine-tuned Witness Gemma 4 E2B judge adapter.

A screenshot of a Google Colab notebook titled "Colab fine-tuning notebook". The subtitle is "Reproducible Unsloth fine-tuning path for the Witness judge." In the top right corner, there are two tabs: "The Witness" (active) and "local-first Gemma 4 firewall". The notebook content is titled "Notebook workflow" and lists six steps: 1. Load Witness judge dataset, 2. Format prompt/response verdict examples, 3. Fine-tune Gemma 4 E2B with Unsloth, 4. Evaluate JSON validity and verdict accuracy, 5. Export model, and 6. Upload/publish model. At the bottom of the workflow section, a URL is provided: <https://colab.research.google.com/drive/17-CgEQLNg8bpnhhWzJwpapRxQyHIqybq?usp=sharing>.

Colab fine-tuning notebook
Reproducible Unsloth fine-tuning path for the Witness judge.

The Witness
local-first Gemma 4 firewall

Notebook workflow

1. Load Witness judge dataset
2. Format prompt/response verdict examples
3. Fine-tune Gemma 4 E2B with Unsloth
4. Evaluate JSON validity and verdict accuracy
5. Export model
6. Upload/publish model

<https://colab.research.google.com/drive/17-CgEQLNg8bpnhhWzJwpapRxQyHIqybq?usp=sharing>

Colab notebook reference for the Unsloth fine-tuning workflow.

ENDPOINT WATCHING

10. Adding an Endpoint

The Blackbox endpoint example demonstrates how a real upstream provider can be watched without storing secrets in the repository.

Set key safely

```
read -s BLACKBOX_API_KEY
export BLACKBOX_API_KEY
```

Add endpoint

```
the-witness endpoint add-blackbox
```

Field	Value
upstream	https://api.blackbox.ai/v1
local proxy	http://localhost:8787/v1
model	blackboxai/x-ai/grok-code-fast-1:free
profile	coding
strictness	high
retry limit	4
auth	bearer_env BLACKBOX_API_KEY

Test through The Witness

```
curl http://localhost:8787/v1/chat/completions \
-H "Authorization: Bearer $BLACKBOX_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "blackboxai/x-ai/grok-code-fast-1:free",
  "messages": [
    {
      "role": "user",
      "content": "Write a Python script that prints Hello World"
    }
  ]
}'
```


Mock endpoint curl test

No external API key is configured, so this usage demo shows the mock endpoint path.

[The Witness](#)[local-first Gemma 4 firewall](#)

terminal - localhost proxy request

```
$ export BLACKBOX_API_KEY="*****" # hidden if used
$ curl http://localhost:8787/v1/chat/completions \
  -H "Authorization: Bearer $BLACKBOX_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "blackboxai/x-ai/grok-code-fast-1:free", "messages": [...]}'

# Through The Witness
{
  "id": "REQ-1024",
  "status": "approved",
  "content": "print(\"Hello World\")"
}
```

Curl test: applications call localhost while The Witness forwards, judges, repairs, and returns approved responses.

STRUCTURED VERDICTS

11. Gemma 4 Judge Schema

```
{
  "verdict": "APPROVED | DISAPPROVED | NEEDS_HUMAN_REVIEW",
  "confidence": 0.0,
  "safety_score": 0,
  "usefulness_score": 0,
  "prompt_alignment_score": 0,
  "correctness_risk": "low | medium | high",
  "rejection_reason": "",
  "suggested_fix": "",
  "improved_prompt_instruction": "",
  "requires_human_review": false
}
```

Field	Purpose
verdict	APPROVED, DISAPPROVED, or NEEDS_HUMAN_REVIEW.
confidence	How confident the judge is in the decision.
safety_score	Safety assessment for the candidate response.
usefulness_score	Whether the answer is practically helpful.
prompt_alignment_score	How well the response follows the original user request.
correctness_risk	Low, medium, or high risk of factual or operational incorrectness.
rejection_reason	Human-readable reason for blocking the answer.
suggested_fix	What must change before retrying.
improved_prompt_instruction	Hidden corrective instruction used for prompt repair.
requires_human_review	Whether The Witness should pause for a person.

REPAIR LOOP

12. Prompt Repair

When Gemma rejects a response, The Witness creates a repaired request that preserves original user intent while adding hidden corrective instructions. It includes the rejection reason, the required fix, and stricter guidance after repeated failures.

Rejected attempt

Original request: Write a Python script that prints Hello World.

```
print(Hello World)
```

DISAPPROVED Python string is not quoted.

Repaired attempt

The previous answer was rejected because the Python string was not quoted. Generate a corrected answer using valid Python syntax.

```
print("Hello World")
```

APPROVED Returned to the app.

Internal details stay internal: prompt repair should not leak secrets or unnecessary judge internals to the final user.

HACKATHON FIT

13. Technology Tracks

Ollama

The default local judge backend runs Gemma 4 through Ollama. The TUI supports gemma4:e2b, gemma4:e4b, and custom Ollama models.

llama.cpp

The project includes a llama.cpp backend for local/resource-constrained inference using Gemma models.

LiteRT

The project includes a LiteRT edge prefilter path for lightweight classification before escalating to a full judge. This path should be treated as experimental unless fully configured.

Unsloth

The public Colab notebook uses Unsloth and the fine-tuned Witness Gemma 4 E2B judge is published as a Hugging Face adapter.

Cactus-ready

The architecture is Cactus-ready for a future mobile companion that routes lightweight checks on-device and heavier checks to the desktop Witness instance. Completed mobile support is not claimed.

Gemma 4 Good Hackathon Track Fit

The Witness is designed around trust, local deployment, and real-world AI reliability.

Main Track

Exceptional vision + real technical execution

Impact

Safety & Trust

Digital Equity & Inclusivity

Health & Sciences

Global Resilience

Technology

Ollama

llama.cpp

LiteRT

Unsloth

Cactus-ready architecture

Primary selected track: Safety & Trust

Track badge wall: The Witness spans Ollama, llama.cpp, LiteRT, Unsloth, Hugging Face, and safety/trust positioning.

IMPACT

14. Impact Tracks

Primary impact track: Safety & Trust. The Witness gives explainable verification, structured verdicts, rejection reasons, prompt repair, human review, and audit logs.

Digital Equity & Inclusivity

Local-first, low-cost, terminal-native, privacy-friendly, low-bandwidth, and multilingual-ready.

Future of Education

AI tutors can be verified before students see answers.

Health & Sciences

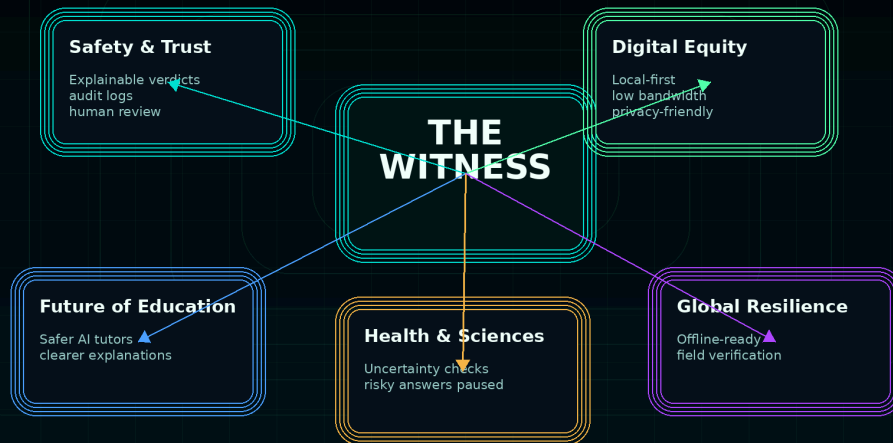
Risky medical/scientific responses can be paused, uncertainty required, and human review triggered.

Global Resilience

Local verification can help field teams and low-connectivity environments where cloud safety tools are unavailable.

Impact Track Map

The Witness is built for places where AI reliability matters most.



Impact map: Safety & Trust is the primary fit, with education, health/sciences, digital equity, and resilience as secondary tracks.

DEMO

15. Demo Walkthrough

1. Install The Witness.
2. Pull gemma4:e2b.
3. Run setup.
4. Start TUI.
5. Add endpoint.
6. Send curl request.
7. The Witness captures the response.
8. Gemma rejects a bad response.
9. Prompt repair happens.
10. Retry succeeds.
11. Audit report is saved.

Demo assets: the repository includes demo_usage/ with screenshots, a usage video, and a usage section. The gallery/ folder contains polished images for Kaggle and GitHub.

Asset	Purpose
demo_usage/full_usage_demo.mp4	Full usage walkthrough video.
demo_usage/screenshots/	Usage screenshots for the writeup/gallery.
gallery/	Polished submission visuals and diagrams.

SECURITY

16. Security and Privacy

- API keys are never stored in docs.
- Examples use environment variables.
- Authorization headers are redacted.
- Privacy mode can avoid storing full prompts/responses.
- Logs can store metadata only.
- The project avoids committing tokens or model weights.

Never paste real API keys into GitHub, screenshots, or videos. Use environment variables such as `$BLACKBOX_API_KEY` and keep local shell history clean when possible.

Secret redaction example

```
Authorization: Bearer *****  
auth: bearer_env BLACKBOX_API_KEY
```

KNOWN BOUNDARIES

17. Limitations

- The Witness reduces risk but does not guarantee correctness.
- Gemma judge decisions can be wrong.
- High-risk medical/legal/financial outputs still need professionals.
- Local performance depends on model and hardware.
- LiteRT may be experimental if not fully tested.
- Cactus/mobile companion is future work unless implemented.
- Streaming support may be limited.
- Fine-tuned model quality depends on training data.

Honest claim: The value of The Witness is a practical reliability layer, not a promise of perfect safety.

VERIFICATION

18. Reproducibility

```
cargo fmt
cargo test
cargo build
bash scripts/verify.sh
the-witness doctor
```

Expected result: build/test should pass. Doctor may warn if Ollama models, Hugging Face tools, external API keys, or optional integrations are missing on the machine.

Before a live demo: run the-witness doctor, make sure the selected backend is reachable, confirm the local proxy port is free, and use demo mode if no external key is configured.

REFERENCES

19. Links

Resource	Link
GitHub repository	https://github.com/PLASMA-FR/the-witness
Fine-tuned model	https://huggingface.co/ahmadalfakeh/witness-gemma4-e2b-judge
Fine-tuning notebook	https://colab.research.google.com/drive/17-CgEQLNg8bpnhhWzJwpapRxQyHlqybq?usp=sharing
Default Ollama tag	<code>gemma4:e2b</code>
Strong Ollama tag	<code>gemma4:e4b</code>

Model names are configurable. The Witness provides practical defaults but lets users enter any custom Ollama model, path, or local OpenAI-compatible judge endpoint.

20. Final Pitch

The Witness is a local-first reliability firewall for the age of AI agents.

As AI systems become more powerful, the question is no longer only:

“Can the model answer?”

The question is:

“Should this answer be allowed to act?”

The Witness turns Gemma 4 into the local trust layer that answers that question.

**Do not just trust AI.
Let The Witness see it
first.**